

DOCUMENT CONVERSION

Non Functional Requirements

Prepared by: Eng Yousef Mahmoud

Email: 7efnaw.ii@gmail.com

The source document content is reproduced in the following pages. No source file was edited or deleted.

Non-Functional Requirements (NFR)

Order Control System (نظام إدارة الطلبات لمطعم سوشي)

1. Performance

- **Response Time:** The system must load the main dashboard and order entry form in under 2 seconds on standard broadband connections.
- **Order Entry Speed:** The UX must be optimized for speed, allowing cashiers to input an order in under 30 seconds during rush hours.
- **Concurrent Users:** The system must comfortably support at least 10 concurrent active sessions without performance degradation (current need is ~3 users, but room for minor expansion).
- **Database Performance:** Complex queries (like end-of-day closing and monthly reports) must execute in under 3 seconds. Appropriate indexes must be used on `Orders` , `Customers` , and `AuditLogs` .

2. Security

- **Authentication:** All user access is guarded by Supabase Auth (Email/Password).
- **Authorization (Backend Enforced):** Role-based access control (RBAC) must be validated on the server side (API routes). Hiding UI elements is insufficient; the API must reject unauthorized actions (e.g., Cashier trying to approve a discount).
- **Data Protection:**
 - Customer data (phone, address) must be securely stored.
 - No hard-delete policy: Database rows for business entities (Orders, Expenses) must never be deleted. Soft deletes (`isActive = false` or `status = CANCELLED`) are mandatory.
- **Audit Trail:** Every critical action (Order creation, status change, cancellation, discount application) must automatically generate an immutable `AuditLog` entry detailing the user, timestamp, and the exact data changes.

3. Usability

- **Bilingual Interface:** The system must fully support both Arabic and English, allowing users to switch languages seamlessly.
- **RTL / LTR Support:** The layout must dynamically adjust for Right-To-Left (Arabic) and Left-To-Right (English) text directions.
- **Responsive Design:** The UI must be fully responsive, prioritized for Tablet (iPad/Android) and Desktop resolutions, as these are the primary devices in the kitchen and management offices.
- **Error Prevention:** Forms must include clear validation (e.g., using Zod) to prevent cashier data entry errors (e.g., missing phone number or selecting a platform without a brand).

4. Reliability

- **Data Integrity:** Financial calculations (Daily Closing, Delivery Fees, Net Cash) must be accurate, isolated in dedicated business logic functions (e.g., `src/lib/closing.ts`), and extensively tested to prevent "monkey business".
- **Backup & Recovery:** Rely on Supabase's automated database backups (Point-in-Time Recovery recommended) to ensure zero data loss in case of severe system failure.
- **Error Handling:** The application must gracefully handle network errors or API timeouts, providing informative error messages to the user rather than blank screens or crashes.

5. Scalability

- **Current Load:** ~100 orders/day across 4 brands, managed by 3 users.
- **Future Growth:** The system architecture (Next.js App Router + Vercel + Supabase) is inherently serverless and scalable. It must be designed to easily accommodate the addition of new brands, platforms, and locations without requiring major database restructuring.

6. Maintainability

- **Code Structure:** Strict Separation of Concerns (SoC). UI components must only handle display and user interaction. Business rules (state machines, calculations, authorization) must reside in shared utility files (`src/lib/`, `src/services/`).
- **Single Source of Truth:** Business rules (e.g., allowed order state transitions) must be defined exactly once in the codebase and imported wherever needed.
- **Documentation:** A strict `PROJECT_LOG.md` policy is enforced. Every structural change, schema update, and business decision must be logged immediately. This acts as a lightweight Architecture Decision Record (ADR).